

Java에서의 Functional 프로그래밍의 현재 상태와 Haskell과의 비교

Jinwoo Jeong

No Institute Given

Abstract. Java는 Object-Oriented 프로그래밍 패러다임을 중심으로 돌아가는 언어입니다. 본격적인 functional 프로그래밍과는 거리가 멀지만 Java 8에서 lambda 함수 등을 추가하는 등 노력을 하고 있습니다. Vavr과 같이 FP 패러다임을 구현하려는 시도를 한 라이브러리도 있지만 Haskell과 같은 functional 프로그래밍 언어를 사용하는 편이 좋아 보입니다.

Keywords: Java · Functional Programming · Lambda Calculus · Haskell

1 Introduction

본 미니 연구는 학술적인 것이 아니며 Java에서의 functional 프로그래밍 구현을 알아보기 위한 것으로 reference로 Wikipedia[13]나 Baeldung[14] 등을 폭 넓게 사용하였으니 참고 시 유의 바랍니다.

Java는 object를 중심으로 돌아가는 Object-Oriented 프로그래밍 언어입니다[7]. 그에 반해 functional 프로그래밍 패러다임은 function을 first-클래스로 취급하며 lambda calculus를 기반으로 하고 있습니다[6]. 모든 method가 class 안에서 생성되는 Java에서 functional interface를 통해 higher-order function을 구현하려는 등 여러 노력이 있었지만 Java를 이용해 본격적인 functional 프로그래밍을 하는 것은 쉽지 않아 보입니다. Vavr 라이브러리는 이러한 Java 환경 속에서 Object-Functional 프로그래밍[3]을 구현하기 위해 만들어졌습니다. 하지만 Java 컴파일러의 근본적인 차이로 인해, side-effect 없는 pure function을 구현하고자 한다면 Haskell과 같은 언어를 사용하는 것이 나아 보입니다.

2 Background

Java에서의 Functional 프로그래밍에 대해 분석하기에 앞서 Functional 프로그래밍이 무엇인지, 그리고 그 기반이 되는 lambda calculus란 어떤지에 대해 개략적으로나마 알 필요가 있습니다.

2.1 Lambda Calculus

Lambda calculus란 computation을 function의 abstraction 및 application을 통해 표현하는 formal system입니다.[4]. Untyped lambda calculus는 모든 종류의

computation을 표현 가능(i.e. Turing-complete)합니다. Typed lambda calculus의 경우 strongly normalizing(i.e. 모든 re-write 시퀀스가 normal form으로 reduce됨)하는 경우 computation이 끝나게 되므로 모든 Turing-computation의 표현이 불가능합니다 [15].

Lambda calculus를 이해하기 위해 lambda term과 transformation 및 reduction에 대해 알아 봅시다. 아래 내용은 Wikipedia를 기반으로 합니다[5][4][16].

$\lambda x.M$ 을 예로 들어봅시다. 해당 expression에는 다음과 같은 lambda term이 있습니다:

- $\lambda x.M$ 는 λ abstraction이라고 부르며, 함수를 정의합니다. λ 는 abstract operator라고 불립니다.
- x 를 variable이라 합니다. Free variable과 bound variable로 구분할 수 있습니다.
- M 은 function의 body입니다. Input 없는 함수의 경우 단순히 M 만으로 표현 될 수가 있습니다.

- $M N$ 의 경우 application이라고 불립니다

예를 들어 $\lambda x.x$ 는 Identity로 입력 값을 그대로 output하며 $\lambda x.x + 1$ 은 입력값 + 1을 주는 function입니다. Variable은 free variable과 bound variable로 분류됩니다. Variable의 substitution 작업의 경우 $M[x := N]$ 의 형태로 표현되며, $\lambda y.M[x := N]$ 의 경우 y 가 N 의 free variable의 set(i.e. $FV(N)$)에 속할 경우 α -renaming이 진행됩니다.

α -equivalence는 bound variable의 이름 변경이 되어도 같은 lambda term이라는 것을 표현 합니다.

β -reduction은 $(\lambda x.t)s$ 형태의 application인 β -redex가 $t[x := s]$ 로 reduce 되는 것입니다. 예시로:

$(\lambda x.x)s \rightarrow \beta x[x := s] \rightarrow s$ 가 있습니다.

Currying은 여러 개의 argument를 받아들이는 function을 단일 argument를 받는 function의 sequence로 만드는 것입니다.

2.2 Functional 프로그래밍

Functional 프로그래밍은 lambda calculus를 구현하며 [6], 컴퓨터의 computation을 수학의 function을 evaluate하는 것과 같이 컴퓨터 프로그램을 작성하는 프로그래밍 패러다임입니다[9].

Functional 프로그래밍 언어의 대표적인 예로는 널리 쓰이는 언어 중 가장 functional하며 이후 Java 코드와 비교에 사용할 Haskell[1]이나 약간의 impurity를 더 하지만 Jane Street가 사용하는 언어로 유명한 OCaml[12] 등이 있습니다.

Functional 프로그래밍의 주요 특징으로는 다음이 있습니다[6][9]:

- Function이 first-class로 취급됨 -> FP의 핵심
- Higher-order Function -> Function을 argument로 받음 OR function을 output 함[17]
- Pure function: Side effect가 없는 함수(메모리나 I/O 등). Haskell에서는 IO 작업에 Monad를 사용함.

언어 구현 상 및 FP로 개발시의 특징으로는 다음이 있습니다:

- Hindley-Milner type 시스템을 이용해 type 표현 -> Haskell 등[19]
- Recursion: Loop의 경우 FP에서는 일반적으로 Recursion을 통해 이루어짐

3 Java에서의 Functional 프로그래밍

3.1 Java와 Lambda Expression

$(f.g.x.f(gx))(+1)(*2)5$ 와 같은 λ expression을 예로 들어봅시다.

Haskell로는 다음과 같습니다.

```
(\ f -> \ g -> \ x -> f (g x)) (+1) (*2) 5
```

Java로는 다음과 같습니다.

```
import java.util.function.Function;
public class LambdaBetaReduction {
    static Function<Function<Integer, Integer>,
        Function<Function<Integer, Integer>,
            Function<Integer, Integer>>> compose =
        f -> g -> x -> f.apply(g.apply(x));

    public static void main(String[] args) {
        int result = compose.apply(n -> n + 1)
            .apply(n -> n * 2)
            .apply(5);
    }
}
```

Java8 이후 추가된 Function interface와 lambda expression을 활용하여야 합니다.

3.2 Immutable 데이터와 pure function에 대해

Referential transparency를 위해선 function이 pure해야 합니다. Function이 pure하기 위해서는 데이터가 immutable 해야합니다.

Haskell의 경우, 기본적으로 모든 데이터가 immutable합니다. List와 같은 데이터

역시 add나 remove 등의 작업이 없습니다[20]. 반면 Java의 record의 경우 기본적으로 class만 immutable하며 내부 object의 경우 재 reference는 불가하지만 그 reference된 주소의 데이터는 변경이 가능합니다.

Java에서 데이터를 immutable하게 하기 위해서는 List.copyOf 등의 방법으로 그러한 데이터의 변경 역시 불가하게 하는 방법으로 일일이 고정시켜줘야 합니다.

4 Java로 Haskell의 feature 따라하기

4.1 Java Optional과 Haskell Maybe

Object의 reference가 null일 수 있는 Java와 달리 Haskell에는 null 타입이 없습니다. 아무 것도 없는 상태를 처리하고 싶을 때에는 Maybe 타입을 사용합니다.

```
data Maybe a = Nothing | Just a
```

Maybe Int 필요한 경우 → Just 5
그런데 없는 경우 → Nothing

Java에서는 Optional을 이용해 비슷하게 표현 가능합니다.

```
Optional<A>
```

Optional<Integer>가 5인 경우 → Optional.of(5)
Optional<Integer>가 없는 경우 → Optional.empty()

4.2 Side Effect에 대하여

Operation이나 expression이 입력을 읽고 value를 return하는 것 외에 외부적인 effect를 가진다면 side effect가 있다고 부릅니다. [21]. I/O 작업 등이 대표적인 예시입니다. Haskell은 IO를 type을 통해 외부와 소통하며, I/O 시스템 자체 역시 purely functional 합니다[22]. 이를 통해 side-effect를 프로그램이 아닌 IO의 실행으로만 한정 하게 됩니다.

Java에서는 I/O를 function으로 감싸는 것 자체는 다음과 같이 흉내가 가능합니다. LaTeX 코드 컴파일 예러가 떠서 이미지로 대체합니다.

```

1  -- getLine  :: IO String
2  -- putStrLn :: String -> IO ()
3
4  greet :: IO ()
5  greet = do
6      putStrLn "이름은?"
7      name <- getLine
8      putStrLn ("내 이름은: " ++ name)

```

(a) Haskell

```

1  import java.util.Scanner;
2  import java.util.function.Function;
3  import java.util.function.Supplier;
4
5  public class PureIo {
6      static Supplier<String> getLine = () -> new Scanner(System.in).nextLine();
7      static Function<String, Supplier<Void>> putStrLn = s -> () -> { System.out.println(s); return null; };
8
9      public static void main(String[] args) {
10         putStrLn.apply("이름은??").get();
11         String name = getLine.get();
12         putStrLn.apply("내 이름은: " + name).get();
13     }
14 }

```

(b) Java

Fig. 1: IO 작업

하지만 function으로 감싸는 것 외에 이것을 구조적으로 Haskell IO처럼 연결 시키고 있지 않습니다. 또 Java는 이것이 컴파일러에서 강제 되지가 않고 그냥 System.out이나 Scanner를 바로 쓸 수 있습니다.

5 결과

Java8에서 Functional 인터페이스와 lambda expression이 추가된 이후, 자바에서 functional 프로그래밍을 하는 것이 한결 편해졌습니다. 하지만, Java의 경우 기본적으로 object-oriented 프로그래밍을 중심으로 한 언어로, functional 프로그래밍을 하기 위해서는 많은 boilerplate이 필요하며, 기본적으로 해당 언어에서 쓰이는 object 및 method를 중심으로 한 프로그래밍과 괴리가 큼니다.

또한, Java에서 Optional, Stream 등으로 비슷하게 표현이 가능한 점도 Haskell 등의 FP 언어와의 컴파일러 상의 차이로 인해 강제되지 않고 확인이 어려운 점이 있습니다. 이로 인해 pure function이나 side-effect free, lazy evaluation도 얼핏 보기엔 똑같이 구현되어 보이지만 내부적으로 또는 그를 받아 이용하는 방식이 근본적으로 차이가 큰 경우가 많습니다.

6 후속 연구

앞서까지 Java에서의 Functional 프로그래밍에 대하여 알아보았습니다. Java와 functional 프로그래밍 및 Haskell과의 비교는 그 내용이 무척 방대하며 구현 내지 비슷한 표현이 됨을 보이는 것은 단순하지만 어렵다는 점을 주장하기에는 추가 조사가 많이 필요해 보입니다. 시간 상의 제약으로 인해 이번 Java에서의 Functional 프로그래밍과 Haskell과의 비교는 큰 틀에서만 진행이 되었습니다.

후에, Java에서 지원이 부족한 Monad들의 연결과 부재한 typeclass, higher-kinded types에 대한 해결법에 대해 알아보려 합니다. Object-Functional 라이브러리라 말하는 Vavr[3]가 이러한 문제를 해결할 수 있는지, 그리고 본 연구에서 지지분하다 말한 Java에서의 functional 프로그래밍을 깔끔히 만들 수 있는지를 위주로 알아보려합니다. 또한, Java 컴파일러가 잡아주지 못 하는 side-effect에 대한 해결 방안이 있는지에 대해서도 조사를 해볼 예정입니다.

또한 본 연구의 주제인 Java에서의 functional 프로그래밍과 별도로, Haskell에서의 다양한 syntax를 typed lambda calculus로 mapping 해 보는 것과 각자 functional 프로그래밍 언어인 Haskell과 OCaml 간의 비교도 진행해 보고 싶습니다.

7 사용된 툴 및 코드

해당 미니 연구를 포함한 GitHub repository는 `git@github.com:kosmostree/mini-researches.git`에서 clone 가능하며 온라인으로도 보실 수 있습니다.

본 연구에서 사용된 Java와 Haskell 툴은 다음과 같습니다.

Java:

- 컴파일러: javac 25.0.3
- Java 버전: openjdk 25.0.3 2026-04-21

Haskell:

- 컴파일러: The Glorious Glasgow Haskell Compilation System, version 9.10.3

References

1. Haskell 홈페이지, <https://www.haskell.org/>, last accessed 2026/07/13
2. GitHub vavr-io/vavr 페이지, <https://github.com/vavr-io/vavr>, last accessed 2026/07/13
3. Vavr 홈페이지, <https://vavr.io/>, last accessed 2026/07/13
4. Wikipedia Lambda calculus 페이지, https://en.wikipedia.org/wiki/Lambda_calculus, last accessed 2026/07/14
5. Wikipedia Lambda calculus definition 페이지, https://en.wikipedia.org/wiki/Lambda_calculus_definition, last accessed 2026/07/14
6. Wikipedia Functional programming 페이지, https://en.wikipedia.org/wiki/Functional_programming, last accessed 2026/07/13

7. Wikipedia Object-oriented Programming 페이지, https://en.wikipedia.org/wiki/Object-oriented_programming, last accessed 2026/07/13
8. Wikipedia Programming paradigm 페이지, https://en.wikipedia.org/wiki/Programming_paradigm, last accessed 2026/07/13
9. Baeldung Functional Programming in Java 페이지, <https://www.baeldung.com/java-functional-programming>, last accessed 2026/07/14
10. Baeldung 페이지, <https://www.baeldung.com/java-8-lambda-expressions-tips>, last accessed 2026/07/14
11. Baeldung Currying in Java 페이지, <https://www.baeldung.com/java-currying>, last accessed 2026/07/14
12. OCaml 웹사이트, <https://ocaml.org/>, last accessed 2026/07/14
13. Wikipedia 웹사이트, <https://www.wikipedia.org/>, last accessed 2026/07/14
14. Baeldung 웹사이트, <https://www.baeldung.com/>, last accessed 2026/07/14
15. Wikipedia Typed lambda calculus 페이지, https://en.wikipedia.org/wiki/Typed_lambda_calculus, last accessed 2026/07/14
16. Wikipedia Currying 페이지, <https://en.wikipedia.org/wiki/Currying>, last accessed 2026/07/15
17. Wikipedia Higher-order function, https://en.wikipedia.org/wiki/Higher-order_function, last accessed 2026/07/15
18. Wikipedia Purely functional data structure 페이지, https://en.wikipedia.org/wiki/Purely_functional_data_structure, last accessed 2026/07/15
19. Wikipedia Hindley Milner type system 페이지, https://en.wikipedia.org/wiki/Hindley%E2%80%93Milner_type_system, last accessed 2026/07/15
20. Haskell.org의 Data.List Docs 페이지, <https://hackage-content.haskell.org/package/base-4.22.0.0/docs/Data-List.html>, last accessed 2026/07/16
21. Wikipedia Side effect 페이지, [https://en.wikipedia.org/wiki/Side_effect_\(computer_science\)](https://en.wikipedia.org/wiki/Side_effect_(computer_science)), last accessed 2026/07/16
22. Haskell.org IO 튜토리얼, <https://www.haskell.org/tutorial/io.html>, last accessed 2026/07/16